Neural Accelerator based on RISC-V with BFloat16 Extensions

Alexander López-Parrado
Facultad de Ingeniería
Universidad del Quindío
Armenia, Colombia
Email: parrado@uniquindio.edu.co

Luis M. Capacho-Valbuena
Facultad de Ingeniería
Universidad del Quindío
Armenia, Colombia
Email: lmcapacho@uniquindio.edu.co

Alexander Vera-Tasamá
Facultad de Ingeniería
Universidad del Quindío
Armenia, Colombia
Email: avera@uniquindio.edu.co

Jorge I. Marín-Hurtado
Facultad de Ingeniería
Universidad del Quindío
Armenia, Colombia
Email: jorgemarin@uniquindio.edu.co

***Abstract*—This paper presents a neural accelerator based on a RISC-V microprocessor whose Instruction Set Architecture (ISA) has been extended with brain floating point (bfloat16) instructions. In this case, we added four new instructions for addition, subtraction, multiplication, and division which were implemented through circuits described in Verilog; each module includes nearest-even rounding circuits to reduce accumulation of errors. The designed neural accelerator allows the implementation of artificial neural networks through the representation of bfloat16 numbers, and it can be used as a coprocessor. To test this functionality, we designed a multiprocessor-based system on an ARM+FPGA System-on-Chip (SoC), where the ARM side has the role of manager, and the RISC-V microprocessor with bfloat16 extensions is the neural coprocessor. Both microprocessors behave as masters, so that they can communicate with each other by using shared memory. The neural accelerator was tested by building a Multilayer Perceptron (MLP) that solves the exlusive or (XOR) problem. This test showed that our RISC-V-based neural accelerator with bfloat16 extensions speeds up the MLP computation $8\times$ compared to the RISC-V microprocessor without bfloat16 extensions.**

***Keywords*—Floating-point arithmetic, artificial neural networks, system-on-chip, hardware acceleration, multiprocessor systems.**

I. Introduction

Artificial intelligence (AI) has been the technology that has revolutionized the world over the past decade, thanks to significant breakthroughs achieved by researchers and private companies in the field of Large Language Models (LLMs). These LLMs have become part of everyday life and provide people with a significant boost in their productivity.

LLMs typically operate in a centralized manner within data centers equipped with computing infrastructure based on Graphics Processing Units (GPUs). These GPUs run sophisticated machine learning algorithms that require enormous computational effort, which translates into high energy consumption, and indirectly contributes to a larger carbon footprint [1].

Considering the above, it is desirable to decentralize LLMs by implementing lightweight versions on edge devices. This would help to reduce energy consumption while still providing the benefits of AI on resource-constrained devices without relying on overly complex infrastructures. For this goal, RISC-V processor seems promising due to its flexibility and scalable nature of its Instruction Set Architecture (ISA). Some efforts have been made to advance in that direction. For instance, authors of [2] make a review of existent AI acceleration architectures based on RISC-V. They also propose a RISC-V-based Convolutional Neural Network (CNN) accelerator by using systolic arrays along with integer arithmetic. In a similar way, authors of [3] and [4] present co-processors tailored to the convolution operation in CNNs. In both works, authors compare co-processor performance against software implementations on scalar RISC-V processors.

In addition, authors of [5] propose two accelerators, one for the convolution operation in CNNs, and another one for the activation function by using the CORDIC algorithm. Both accelerators use 32-bit floating-point arithmetic and were tested along with a RISC-V processor.

Similarly, authors of [6] designed RISC-V instruction extensions to detect driver drowsiness. In this case, authors developed extensions targeting Multiply-Accumulate (MAC) operation to accelerate CNNs, for which they used dynamic fixed-point arithmetic instead of floating-point arithmetic.

Furthermore, in [7] authors proposed a CNN accelerator based on Single Instruction Multiple Data (SIMD) extensions of a RISC-V microprocessor. In this case, authors use the Fermat Number Transform (FNT) to accelerate the convolution operation on images.

Continuing, authors of [8] developed the so-called Neural Learning Unit (NLU) that incorporates 20 instructions based on brain floating-point (bfloat16) format to accelerate the convolution operation and activation functions of CNNs. In addition, the works in [9] and [10] present multi-format Floating Point Units (FPU) to extend the ISA of RISC-V.

On the other hand, the work in [11] proposes a compact RISC-V implementation including a low-latency mechanism

to handle interrupts. Authors claim this feature makes their design suitable to control AI accelerators. Finally, in [12] authors propose a RISC-V-based barrel processor capable of scheduling 8 hardware threads, each processing element incorporates hardware to perform GEneral Matrix Vector (GEMV) product operation.

In summary, most of reviewed works address acceleration of convolution operation in CNNs, some works focused on reducing latency or increasing parallelism of the RISC-V architectures. However, few works deal with the problem of extending the RISC-V ISA with bfloat16 instructions and their application on AI.

This work addresses the problem of extending the RISC-V ISA with bfloat16 instructions. Fig. 1 illustrates the structure of the bfloat16 format, a 16-bit floating-point representation developed by Google to lighten computer effort in machine learning applications [13].

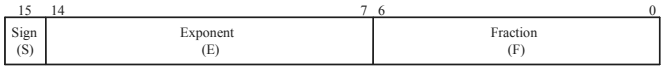

Fig. 1. BFloat 16 number format.

The format comprises a 1-bit sign (S), an 8-bit exponent (E), and a 7-bit fraction or mantissa (F). Unlike the IEEE 754 half-precision format, bfloat16 maintains the same exponent width as single-precision (float32), enabling a similar dynamic range while reducing memory and computing requirements [14].

The remaining of the paper is organized as follows: Section II describes the neural accelerator architecture, Section III presents the tests performed on the neural accelerator, and Section IV the final remarks of this paper.

II. Dual-processor Acceleration Architecture

We designed a dual-processor heterogeneous architecture integrating an ARM core and a RISC-V core based on a Zynq SoC [15]. The proposed architecture is shown in Fig. 2.

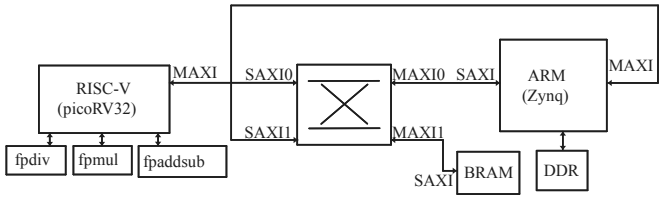

Fig. 2. Multiprocessor architecture.

The ARM core is a hard processor in the Zynq SoC and the RISC-V core represents the neural accelerator, which is implemented on the FPGA side of the Zynq SoC. In this case, the RISC-V core is equipped with bfloat16 extensions for addition/subtraction, multiplication and division aiming the acceleration of ANN operations.

The system utilizes AXI (Advanced eXtensible Interface) bridges to connect master and slave components, enabling seamless data exchange. Both ARM and RISC-V cores perform inter-processor communication via shared memory residing in DDR memory starting at the physical address `0x10000000`. The block RAM (BRAM) is allocated for storing the RISC-V core program.

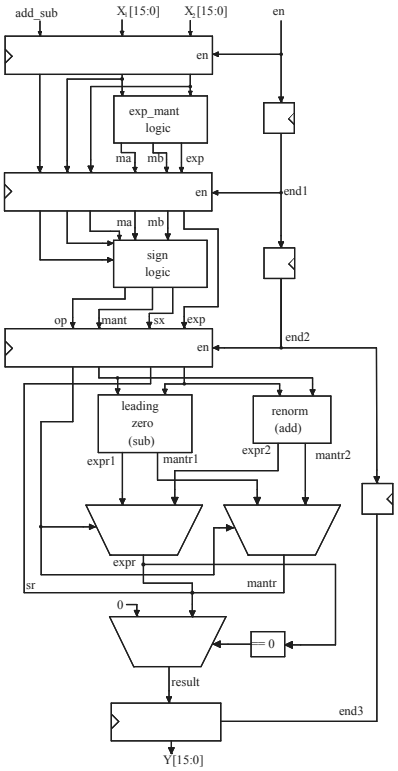

Fig. 3. Pipeline for addition/subtraction instruction.

A. BFloat16 Instructions Implementation

We used the picorv32 core which implements the RV32I instruction set [16]. It was enhanced with custom instructions by bfloat16 extensions for addition/subtraction, multiplication and division.

Custom instructions use the Pico Co-Processor Interface (PCPI) available in picorv32 [17]. In this way, 4 new instructions were added (bf16add, bf16sub, bf16mul and bf16div) through 3 architectures. The bf16add and bf16sub instructions are supported by a unique pipelined architecture. Fig. 3 depicts the pipelined architecture for the bfloat16 adder/subtractor.

The circuit processes two 16-bit bfloat16 operands, X_1 and X_2, through a two-stage pipeline along with input and output registers. The `add_sub` signal selects performed operation, addition or subtraction. The architecture comprises exponent and mantissa extraction, sign logic, leading-zero detection for subtraction, and renormalization for addition. Our design includes the block `exp_mant_logic` which simultaneously performs mantissa alignment and nearest-even rounding. Fig. 4 illustrates the pipelined architecture for the bfloat16 multiplier.

The circuit accepts two 16-bit bfloat16 inputs (X_1 and X_2), which are first processed through a stage that extracts and computes the sign, exponent, and mantissa components. The

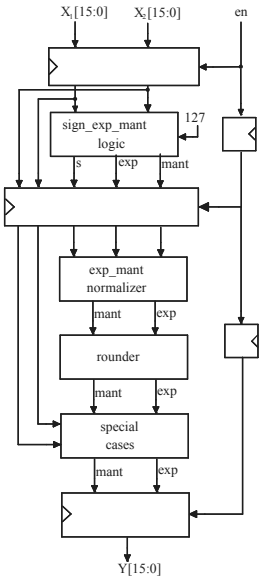

Fig. 4. Pipeline for multiplication instruction.

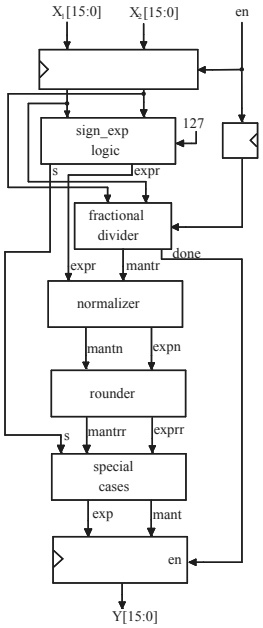

Fig. 5. Serial circuit for division instruction.

pipeline includes logic for exponent and mantissa normalization, and nearest-even rounding. Special cases such as zero, infinity, and NaN are also handled explicitly. The final product (Y) is produced in bfloat16 format. Fig. 5 illustrates the serial architecture for the bfloat16 divider.

The circuit accepts two 16-bit bfloat16 inputs (X_1 and X_2), which are first processed through a stage that extracts and computes the sign and exponent. In this case, the mantissa is computed through a bit-serial divider, after that, the circuit performs exponent and mantissa normalization and nearest-even rounding. Special cases such as zero, infinity, and NaN are also handled explicitly. The final product (Y) is produced in bfloat16 format.

The above architectures balance latency and hardware complexity, making them well-suited for high-throughput floating-point operations in machine learning accelerators. Table I depicts computation time in clock-cycles for each bfloat16 circuit.

TABLE I
CLOCK-CYCLES FOR EACH CIRCUIT

Circuit	Clock cycles
Adder/subtractor	4
Multiplier	3
Divider	14

In particular, adder/subtractor takes 4 clock cycles to complete its operation, multiplier takes 3 clock cycles and divider takes 14 clock cycles, which is due to the bit-serial mantissa divider.

Implementation of bfloat16 extensions is completed at firmware side by using the RISC-V instruction R-format along with the custom-1 opcode (`0x2B`) and inline assembly [16], this approach saved us from recompiling the GNU C Compiler (GCC) toolchain.

III. TESTS AND RESULTS

The neural accelerator was tested by implementing the architecture in Fig. 2 on a Zynq SoC [15]. In this case, the neural accelerator is clocked at a frequency of 100 MHz.

A. FPGA Synthesis Results

Table II shows the utilization report of FPGA resources for the neural accelerator.

TABLE II
UTILIZATION REPORT

Resource	Utilization
Flip-Flops	912/35200
LUTs	1399/17600

From utilization report, it can be concluded that our neural accelerator uses around 2.6% of available flip-flops and 7.95% of available combinatorial look-up tables (LUTs). On the other hand, Fig. III presents the timing report after synthesis.

TABLE III
TIMING REPORT

Parameter	Value
Worst Negative Slack (WNS)	2.852ns
Worst Hold Slack (WHS)	0.007ns
Worst Pulse Width Slack (WPWS)	3.750

Timing report in Table III shows a worst negative slack of 2.852 ns, which means our neural accelerator can operate using a clock signal with frequency of upto $1/(10ns -$

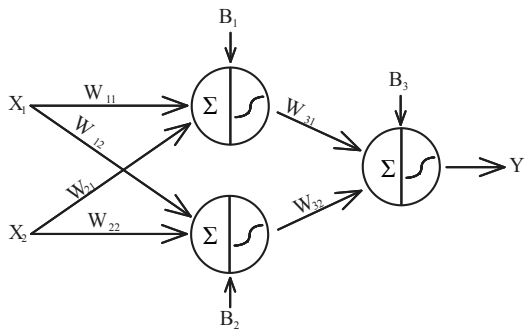

Fig. 6. Multilayer perceptron for XOR problem.

$2.852ns) = 139.89MHz$. Hence, by considering reached maximum frequency and values in Table I, throughput of designed bfloat16 circuits is presented in Table IV.

TABLE IV
THROUGHPUT FOR EACH CIRCUIT

Circuit	Throughput
Adder/subtractor	35 MFLOPS
Multiplier	47 MFLOPS
Divider	10 MFLOPS

From Table IV, it can be seen that fastest bfloat16 circuit is the multiplier and the slowest bfloat16 circuit is the divider. Nonetheless, it is important to notice division is not a quite common operation in AI as additions and multiplications.

B. XOR Gate Neural Network

After finishing synthesis and implementation on FPGA, we coded a firmware implementing a Multilayer Perceptron (MLP) that solves the XOR problem shown in Fig. 6 [18].

Weigths and biases of above MLP were trained by using a Python script where bfloat16 format was considered. In this case, we used the sigmoid activation function shown in Eq. 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

In particular, $\sigma(x)$ was implemented at firmware level through a LUT.

Furthermore, we coded two versions of the MLP in Fig. 6. First version uses designed bfloat16 extensions, and second version doesn't use bfloat16 extensions but a software implementation of the arithmetic operations. In-system test results showed the first version outperformed the speed of second version by $8\times$ when evaluating the MLP in Fig. 6.

IV. CONCLUSION

This paper presented a neural accelerator based on a RISC-V microprocessor extended with custom bfloat16 instructions. The accelerator was implemented as a coprocessor in a heterogeneous ARM+FPGA SoC architecture, demonstrating significant performance improvements for AI workloads.

Consequently, we created four bfloat16 instructions (addition, subtraction, multiplication, and division) which were implemented using Verilog modules with nearest-even rounding, maintaining computational accuracy while optimizing hardware efficiency.

In addition, a dual-processor system was developed where an ARM core manages tasks while the RISC-V coprocessor handles neural network computations. The design leverages shared memory and AXI bridges for efficient inter-processor communication.

Our system was tested with an XOR-solving MLP showing an $8\times$ speedup compared to a software-only implementation on the base RISC-V processor, validating the effectiveness of the bfloat16 hardware acceleration.

Finally, future work will focus on expanding the instruction set to support vector operations and exploring Processing In-Memory (PIM) acceleration. The modular design also permits integration with other RISC-V cores, making it adaptable for diverse edge-AI scenarios. This work contributes to the growing ecosystem of RISC-V-based AI accelerators, offering a power-efficient solution for decentralized machine learning.

ACKNOWLEDGMENT

Authors thank *Universidad del Quindío* for supporting this project.

REFERENCES

[1] Z. Chen, W. Lin, X. Xie, Y. Hu, C. Li, Q. Tong, Y. Wu, and S. Li, "An empirical study on the power consumption of llms with different gpu platforms," in *2024 IEEE International Conference on Big Data (BigData)*, 2024, pp. 8640–8642.

[2] A. Sanchez-Flores, L. Alvarez, and B. Alorda-Ladaria, "Accelerators in embedded systems for machine learning: A riscv view," in *2023 38th Conference on Design of Circuits and Integrated Systems (DCIS)*, 2023, pp. 1–6.

[3] S. K. Mousavikia and M. Mousazadeh, "Comparison between hardware/software co-design of riscv vector and scalar implementation of deep neural networks," in *2024 6th Iranian International Conference on Microelectronics (IICM)*, 2024, pp. 1–5.

[4] M. F. Tolba, H. Saleh, M. Al-Qutayri, A. Hroub, and T. Stouraitis, "Efficient cnn hardware architecture based on linear approximation and computation reuse technique," in *2023 International Conference on Microelectronics (ICM)*, 2023, pp. 7–10.

[5] D.-Z. Li, H.-R. Gong, and Y.-C. Chang, "Implementing riscv system-on-chip for acceleration of convolution operation and activation function based on fpga," in *2018 14th IEEE International Conference on Solid-State and Integrated Circuit Technology (ICSICT)*, 2018, pp. 1–3.

[6] S. K. Mousavikia, E. Gholizadehazari, M. Mousazadeh, and S. B. O. Yalcin, "Instruction set extension of a riscv based soc for driver drowsiness detection," *IEEE Access*, vol. 10, pp. 58 151–58 162, 2022.

[7] B. Chen, X. Wang, Y. Huang, and Z. Xu, "Riscv-fnt: A fast fnt-based risc-v processor for cnn acceleration," in *2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)*, 2024, pp. 292–296.

[8] A. Rostami, S. M. A. Zeinolabedin, L. Guo, F. Kelber, H. Bauer, A. Dixius, S. Scholze, M. Berthel, D. Walter, J. Uhlig, B. Vogginger, and C. Mayr, "Nlu: An adaptive, small-footprint, low-power neural learning unit for edge and iot applications," *IEEE Open Journal of Circuits and Systems*, vol. 6, pp. 85–99, 2025.

[9] K. Yang and J. F. Martínez, "Floatap: Supporting high-performance floating-point arithmetic in associative processors," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024, pp. 657–670.

[10] S. Mach, F. Schuiki, F. Zaruba, and L. Benini, "Fpnew: An open-source multiformat floating-point unit architecture for energy-proportional transprecision computing," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 4, pp. 774–787, 2021.

[11] Y. Cui, J. Xiong, Z. Yang, and L. Zhang, "A tiny processor with rapid interrupt response for ai accelerator control," in *2024 4th International Conference on Electronic Information Engineering and Computer Communication (EIECC)*, 2024, pp. 1281–1284.

[12] M. AskariHemmat, O. Bilaniuk, S. Wagner, Y. Savaria, and J.-P. David, "Risc-v barrel processor for deep neural network acceleration," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021, pp. 1–5.

[13] S. Wang and P. Kanwar, "BFloat16: The secret to high performance on Cloud TPUs," https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus, 2019, [Online; accessed 22-apr-2025].

[14] "Ieee standard for floating-point arithmetic," *IEEE Std 754-2019 (Revision of IEEE 754-2008)*, pp. 1–84, 2019.

[15] Advanced Micro Devices, Inc., *Zynq-7000 All Programmable SoC Technical Reference Manual*, 2021, uG585. [Online]. Available: https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM

[16] RISC-V International, *The RISC-V Instruction Set Manual Volume I Unprivileged Architecture*, April 2024, available at https://github.com/riscv/riscv-isa-manual/releases/download/riscv-isa-release-5308687-2025-04-22/riscv-privileged.pdf.

[17] Yosys Headquarters, "PicoRV32 - A Size-Optimized RISC-V CPU," https://github.com/YosysHQ/picorv32/, 2019, [Online; accessed 22-apr-2025].

[18] C. C. Aggarwal, *Neural Networks and Deep Learning*, 2nd ed. Springer, 2023. [Online]. Available: https://doi.org/10.1007/978-3-031-29642-0